Swift — SwiftfulThinkingBasicsBootcamp Source Code

SwiftBasics.xcplaygroundpage/Contents.swift

```swift
import Foundation

var greeting = "Hello, playground"

print(greeting)
print(greeting)
print(greeting)
print(greeting)
print(greeting)


// This is a single-line comment

// These are multiple single-line comments
// This is a comment
// and this is some more info

/*
 This is a multi-line comment
 that goes to
 mutliple lines!
 */


// Naming Conventions

// Camel Case - CORRECT!
// The first word is lowercased and then the first character in every following word is uppercased.

let firstGreeting = "Hello, world!"
let thisIsMyFirstGreeting = "Hello, world!"

// wrong
let thisismysecondgreeting = "Hello, world!"

// Snake Case - wrong
let this_is_what_snake_case_looks_like = "Hello, world!"

// Pascal Case - wrong
let ThisIsWhatPascalCaseLooksLike = "Hello, world!"

// Camel Case - right
let thisIsWhatCamelCaseLooksLike = "Hello, world!"
```

BasicTypes.xcplaygroundpage/Contents.swift

```swift
import Foundation

// String is "regular text"
let myFirstItem: String = "Hello, world!"

// Reference previously created objects
let myTitle = myFirstItem

// Boolean (aka Bool) is true or false
let mySecondItem: Bool = true
let myThirdItem: Bool = false

// Do not do!
//let myFourthItem: String = "true"

// Swift is a type-safe language
let myFifthItem: Bool = true
let mySixthItem: String = "Hello, world!"
let mySeventhItem = "Hello, world!"

// Date
let myFirstDate: Date = Date()


// Number can be Int, Double, CGFloat & more

// Int is a whole Integer
let myFirstNumber: Int = 1

// Use Double for math
let mySecondNumber: Double = 1.0

// Use CGFloat for UI
let myThirdNumber: CGFloat = 1.0
```

Variables.xcplaygroundpage/Contents.swift

```swift
import Foundation


// Constant
let someConstant: Bool = true

// Variable
var someVariable: Bool = true

// Cannot assign to value: 'someConstant' is a 'let' constant
// someConstant = false

someVariable = false

var myNumber: Double = 1.1234
print(myNumber)
myNumber = 2
print(myNumber)
myNumber = 234870234
print(myNumber)
myNumber = 1
print(myNumber)
myNumber = 458


// if statements

var userIsPremium: Bool = false

if userIsPremium == true {
    print("1 - user is premium")
} else {
    print("1.1 - user is NOT premium")
}

if userIsPremium {
    print("2 - user is premium")
}

if userIsPremium == false {
    print("3 - user is NOT premium")
}

if !userIsPremium {
    print("4 - user is NOT premium")
}
```

Operators.xcplaygroundpage/Contents.swift

```swift
import Foundation

//var likeCount: Double = 5
//var commentCount: Double = 0
//var viewCount: Double = 100

//likeCount = 5 + 1

// Addition
//likeCount = likeCount + 1
//likeCount += 1

// Subtraction
//likeCount = likeCount - 1
//likeCount -= 1

// Multiplication
//likeCount = likeCount * 1.5
//likeCount *= 1.5

// Division
//likeCount = likeCount / 2
//likeCount /= 2

// Order of operations does matter!
// PEMDAS
//likeCount = likeCount - 1 * 1.5
//likeCount = (likeCount - 1) * 1.5


var likeCount: Double = 5
var commentCount: Double = 0
var viewCount: Double = 100

likeCount -= 1

print(likeCount)

if likeCount == 5 {
    print("Post has 5 likes.")
} else {
    print("Post does NOT have 5 likes.")
}

if likeCount != 5 {
    print("Post does NOT have 5 likes.")
}

if likeCount > 5 {
    print("Post has greater than 5 likes.")
}

if likeCount >= 5 {
    print("Post has greater than or equal to 5 likes.")
}

if likeCount < 5 {
    print("Post has less than 5 likes.")
}

if likeCount <= 5 {
    print("Post has less than or equal to 5 likes.")
}

if (likeCount > 3) && (commentCount > 0) {
    print("Post has greater than 3 likes AND greater than 0 comments.")
} else {
   print("Post has 3 or less likes OR post has 0 or less comments.")
}

if likeCount > 3 || commentCount > 0 {
    print("Post has greater than 3 likes OR greater than 0 comments.")
} else {
    print("Post has 3 or less likes AND post has 0 or less comments.")
}

var userIsPremium: Bool = true
var userIsNew: Bool = false

if userIsPremium && userIsNew {

}
```

```swift
if (likeCount > 3 && commentCount > 0) || (viewCount > 50) {
    print("EXECUTE")
}

if (likeCount > 100) && (commentCount > 0 || viewCount > 50) {
    print("EXECUTE")
}

if likeCount > 5 || userIsPremium {

}


if likeCount > 5 {
    print("Like count > 5")
} else if likeCount > 2 {
    print("Like count > 2")
} else if userIsPremium {
    print("user is premium")
} else {
    print("else statement")
}
```

Functions.xcplaygroundpage/Contents.swift

```swift
import Foundation


func myFirstFunction() {
    print("MY FIRST FUNCTION CALLED")
    mySecondFunction()
    myThirdFunction()
}

func mySecondFunction() {
    print("MY SECOND FUNCTION CALLED")
}

func myThirdFunction() {
    print("MY THIRD FUNCTION CALLED")
}

myFirstFunction()


func getUserName() -> String {
    let username: String = "Nick"
    return username
}

func checkIfUserIsPremium() -> Bool {
    return false
}

let name: String = getUserName()



// ------------------------------------------

showFirstScreen()

func showFirstScreen() {
    var userDidCompleteOnboarding: Bool = false
    var userProfileIsCreated: Bool = true
    let status = checkUserStatus(didCompleteOnboarding: userDidCompleteOnboarding, profileIsCreated:
userProfileIsCreated)

    if status == true {
        print("SHOW HOME SCREEN")
    } else {
        print("SHOW ONBOARDING SCREEN")
    }
}

func checkUserStatus(didCompleteOnboarding: Bool, profileIsCreated: Bool) -> Bool {
    if didCompleteOnboarding && profileIsCreated {
        return true
    } else {
        return false
    }
}

func doSomethingElse(someValue: Bool) {

}

// ------------------------------------------------------

let newValue = doSomething()

func doSomething() -> String {
    var title: String = "Avengers"

    // "If title is equal to Avengers"
    if title == "Avengers" {
        return "Marvel"
    } else {
        return "Not Marvel"
    }
}

checkIfTitleIsAvengers()


func checkIfTitleIsAvengers() -> Bool {
    var title: String = "Avengers"
```

```
    // "Make sure title == Avengers
    guard title == "Avengers" else {
        return false
    }

    return true
}

func checkIfTitleIsAvengers2() -> Bool {
    var title: String = "Avengers"

    if title == "Avengers" {
        return true
    } else {
        return false
    }
}



// Calculated variables are basically functions
// Generally good for when you don't need to pass data into the function

let number1 = 5
let number2 = 8

func calculateNumbers() -> Int {
    return number1 + number2
}

func calculateNumbers(value1: Int, value2: Int) -> Int {
    return value1 + value2
}

let result1 = calculateNumbers()
let result2 = calculateNumbers(value1: number1, value2: number2)

var calculatedNumber: Int {
    return number1 + number2
}
```

Optionals.xcplaygroundpage/Contents.swift

```swift
import Foundation



// "There is always a value and it is a Boolean"
let myBool: Bool = false

// "We don't know if there is a value, but if there is, it is a Boolean"
var myOtherBool: Bool? = nil


// we don't know if there is a value, but if there is, it is a Boolean
let myOtherBool: Bool? = false
print(myOtherBool) // Optional(false)

var myOtherBool2: Bool? = nil
print(myOtherBool2) // nil

myOtherBool2 = true
print(myOtherBool2) // Optional(true)
print(myOtherBool2!) // true

// also optional bool
let newValue: Bool? = myOtherBool2

// the value of myOtherBoo12 (if there is one), otherwise false
myOtherBool2 = nil
let newValue2: Bool = myOtherBool2 ?? false
print("New value 2: \(newValue2.description)")
print("New value 2: \(newValue2)")

// String also appy the optional
var myString: String? = nil
let newString = myString ?? "Hello" // if myString has no value, it return "Hello"
print(newString)

myString = "Hello World" // it has the value now
print(myString ?? "No value") // so it will print the "Hello World"


// ------------------------------------------------


var userIsPremium: Bool? = nil

// return the boolean
func checkIfUserIsPremium() -> Bool? {
    return userIsPremium
}

// this function make sure that it return with the boolean
// since it default value will be false
func checkIfUserIsPremium2() -> Bool {
    return userIsPremium ?? false
}

let isPremium = checkIfUserIsPremium2()


// If-let
// When if-let is successful, enter the closure
func checkIfUserIsPremium3() -> Bool {

    // If there is a value, let newValue equal that value
    // if there is value in the userIsPreminum, since userIsPremium is Bool?
    if let newValue = userIsPremium {
        // Here we have access to the non-optional value
        return newValue
    } else {
        return false
    }
}

// same as the checkIfUserIsPremium3
func checkIfUserIsPremium4() -> Bool {
    if let newValue = userIsPremium {
        return newValue
    }

    return false
}
```

```swift
// same as the checkIfUserIsPremium3
func checkIfUserIsPremium5() -> Bool {
    if let userIsPremium {
        return userIsPremium
    }

    return false
}


// Guard
// When a guard is a failure, enter the closure
func checkIfUserIsPremium6() -> Bool {

    // Make sure there is a value
    // If there is, let newValue equal that value
    // Else (otherwise) return out of the function
    guard let newValue = userIsPremium else {
        return false
    }

    // Here we have access to the non-optional value
    return newValue
}

func checkIfUserIsPremium7() -> Bool {
    guard let userIsPremium else {
        return false
    }

    return userIsPremium
}


// ----------------------------------------------------


var userIsNew: Bool? = true
var userDidCompleteOnboarding: Bool? = false
var userFavoriteMovie: String? = nil

// this function is to check that the variable is no longer optional
func checkIfUserIsSetUp() -> Bool {

    // unwrap
    if let userIsNew, let userDidCompleteOnboarding, let userFavoriteMovie {
        // userIsNew == Bool AND
        // userDidCompleteOnboarding == Bool AND
        // userFavoriteMovie == String

        // pass the value to another function (getUserStatus)
        return getUserStatus(
            userIsNew: userIsNew,
            userDidCompleteOnboarding: userDidCompleteOnboarding,
            userFavoriteMovie: userFavoriteMovie
        )
    } else {
        // userIsNew == nil OR
        // userDidCompleteOnboarding == nil OR
        // userFavoriteMovie == nil
        return false
    }
}


func checkIfUserIsSetUp2() -> Bool {

    guard let userIsNew, let userDidCompleteOnboarding, let userFavoriteMovie else {
        // userIsNew == nil OR
        // userDidCompleteOnboarding == nil OR
        // userFavoriteMovie == nil
        return false
    }

    // userIsNew == Bool AND
    // userDidCompleteOnboarding == Bool AND
    // userFavoriteMovie == String
    return getUserStatus(
        userIsNew: userIsNew,
        userDidCompleteOnboarding: userDidCompleteOnboarding,
        userFavoriteMovie: userFavoriteMovie
    )
}

func getUserStatus(userIsNew: Bool, userDidCompleteOnboarding: Bool, userFavoriteMovie: String) -> Bool {
    if userIsNew && userDidCompleteOnboarding {
        return true
```

```
    }

    return false
}


// layered if-let
// more complex to do this
func checkIfUserIsSetUp3() -> Bool {
    if let userIsNew {
        // userIsNew == Bool

        if let userDidCompleteOnboarding {
            // userDidCompleteOnboarding == Bool

            if let userFavoriteMovie {
                // userFavoriteMovie == String
                return getUserStatus(
                    userIsNew: userIsNew,
                    userDidCompleteOnboarding: userDidCompleteOnboarding,
                    userFavoriteMovie: userFavoriteMovie
                )
            } else {
                // userFavoriteMovie == nil
                return false
            }

        } else {
            // userDidCompleteOnboarding == nil
            return false
        }
    } else {
        // userIsNew == nil
        return false
    }
}


// layered guard
// so in this case that guard is more useful
func checkIfUserIsSetUp4() -> Bool {
    guard let userIsNew else {
        // userIsNew == nil
        return false
    }
    // userIsNew == Bool

    guard let userDidCompleteOnboarding else {
        // userDidCompleteOnboarding == nil
        return false
    }
    // userDidCompleteOnboarding == Bool

    guard let userFavoriteMovie else {
        // userFavoriteMovie == nil
        return false
    }
    // userFavoriteMovie == String

    return getUserStatus(
        userIsNew: userIsNew,
        userDidCompleteOnboarding: userDidCompleteOnboarding,
        userFavoriteMovie: userFavoriteMovie
    )
}

func checkIfUserIsSetUp5() -> Bool {
    guard let userIsNew else {
        return false
    }

    guard let userDidCompleteOnboarding else {
        return false
    }

    guard let userFavoriteMovie else {
        return false
    }

    return getUserStatus(
        userIsNew: userIsNew,
        userDidCompleteOnboarding: userDidCompleteOnboarding,
        userFavoriteMovie: userFavoriteMovie
    )
}

// Optional chaining
```

```swift
func getUsername() -> String? {
    return "test"
}

func getTitle() -> String {
    return "title"
}

func getUserData() {


    let username: String? = getUsername()

    // "I will get the count if the username is not nil"
    // this count is also optional
    let count: Int? = username?.count

    let title: String = getTitle()

    // "I will get the count always"
    let count2 = title.count



    // If username has a value, and first character in username has a value, then return the value of isLowercase
    // Optional chaining
    let firstCharacterIsLowercased = username?.first?.isLowercase ?? false

    // "If will get the count because I know 100% that username is not nil"
    // This will crash your application if username is nil!
    let count3: Int = username!.count

}

// safely unwrap an optional
// nil coalscing
// if-let
// guard


// explicitly unwrap optional
// !
```

Tuples.xcplaygroundpage/Contents.swift

```swift
import Foundation



var userName: String = "Hello"
var userIsPremium: Bool = false
var userIsNew: Bool = true

// return the userName directly
func getUserName() -> String {
    userName
}
func getUserIsPremium() -> Bool {
    userIsPremium
}

// limited to 1 return type
func getUserInfo() -> String {

    let name = getUserName()
    let isPremium = getUserIsPremium()

    return name
}

// tuple can combine multiple pieces of data
func getUserInfo2() -> (String, Bool) {
    let name = getUserName()
    let isPremium = getUserIsPremium()

    return (name, isPremium)
}

var userData1: String = userName
var userData2: (String, Bool, Bool) = (userName, userIsPremium, userIsNew)


let info1 = getUserInfo2()
// 0 means the order in the info1
let name1: String = info1.0
// let name1: String = info1.1 // we cannot use .1, since info1.1 is the Boolean not the String

func getUserInfo3() -> (name: String, isPremium: Bool) {
    let name = getUserName()
    let isPremium = getUserIsPremium()

    return (name, isPremium)
}

// info2 is the tuple
let info2 = getUserInfo3()
// here the index is no longer the number, is the variable now
let name2 = info2.name
let isPremium = info2.isPremium

func getUserInfo4() -> (name: String, isPremium: Bool, isNew: Bool) {
    return (userName, userIsPremium, userIsNew)
}

func doSomethingWithUserInfo(info: (name: String, isPremium: Bool, isNew: Bool)) {

}


let info = getUserInfo4()
// the first info is the parameters in the doSomethingWithUserInfo, and the second info is the let variable
doSomethingWithUserInfo(info: info)
```

ObjectOrientedProgramming.xcplaygroundpage/Contents.swift

```swift
import Foundation


/*

 // Object Oriented Programming

 // During the life the app, we create and destroy objects
 // Create = Initialize (init) = Allocate (add to memory)
 // Destroy = Deinitialize (deinit) = Deallocate (remove from memory)

 // Automatic Reference Counting (ARC)
 // A live count of the number of objects in memory
 // Create 1 object, count goes up by 1
 // Create 2 objects, count goes up by 2
 // Destroy 1 object, count goes down by 1

 // The more objects in memory, the slower the app performs
 // We want to keep the ARC count as low as possible
 // We want to create objects only when we need them
 // And destory them as soon as we no longer need them

 // For example, if an app has 2 screens and user is moving from screen 1 to screen 2.
 // We only want to allocate screen 2 WHEN we need it (ie. when the user clicks a button to segue to screen 2).
When we get to screen 2, we may want to deallocate screen 1.


 // There are 2 types of Memory
 // Stack & Heap
 // Only objects in the Heap are counted towards ARC


 // Advanced info here:
 // https://www.youtube.com/watch?v=-JLenSTKEcA&themeRefresh=1


 // Objects in the Stack
 // String, Bool, Int, most basic types
 // NEW: Struct, Enum, Array

 // Objects in the Heap
 // Functions
 // NEW: Class, Actors
 // Only above will be counted in ARC


 // iPhone is a "multi-threaded environment"
 // There are multiple "threads" or "engines" running simultaneously
 // Each threads has a Stack
 // But there is only 1 Heap for all threads

 // Therefore:
 // Stack is faster, lower memory footprint, preferable // this is the reason why we use struct in SwiftUI
 // Heap is slower, higher memory footprint, risk of threading issues


 // Value vs Reference types

 // Objects in the Stack are "Value" types.
 // When you edit a Value type, you create a copy of it with new data.

 // Objects in the Heap are "Reference" types.
 // When you edit a Reference type, you edit the object that you are referencing. This "reference" is called
"pointer" because it "points" to an object in the Heap (in memory).

 */


struct MyFirstObject {
    let title: String = "Hello, world!"
}

class MySecondObject {
    let title: String = "Hello, world!"
}


// Class vs Struct explained to a 5-year old

// Imagine a school and in the school there are classrooms.
// Within each class, there are quizzes.
// During the day, the teacher will hand out many different quizzes to different classes. The students will answer
the quizzes and return them back to the teacher.
```

```
// "school" = App
// "classroom" = Class
// "quiz" = Struct

// In this example, we have a classroom and there are many actions that occur inside the classroom.
// In code, we create a class and can perform actions within the class.

// In this example, there are many different types of quizzes. The teacher hands out the quizzes and the students
take the quizzes and return them to the teacher.
// In code, we create many structs and pass them around out app with ease.

// Note:
// This metaphor is NOT perfect :)
// Technical a "quiz" can be a class, etc.


// We want to use a class for things like:
// "Manager" "DataService" "Service" "Factory" "ViewModel"
// Objects that we create and want to perform actions inside.

// We want to use a struct for things like:
// Data models
// Objects that we create and pass around our app.
```

Structs.xcplaygroundpage/Contents.swift

```
import Foundation


// Structs are fast!
// Structs are stored in the Stack (memory)
// Objects in the Stack are Value types
// Value types are copied & mutated


struct Quiz {
    let title: String
    let dateCreated: Date
    let isPremium: Bool?

    // Structs have an implicit init
//    init(title: String, dateCreated: Date) {
//        self.title = title
//        self.dateCreated = dateCreated
//    }

//    init(title: String, dateCreated: Date = .now) {
//        self.title = title
//        self.dateCreated = dateCreated
//    }

    init(title: String, dateCreated: Date?, isPremium: Bool?) {
        self.title = title
        self.dateCreated = dateCreated ?? .now
        self.isPremium = isPremium
    }
}


let myObject: String = "Hello, world!"

//let myQuiz: Quiz = Quiz(title: "Quiz 1", dateCreated: .now)
//let myQuiz: Quiz = Quiz(title: "Quiz 1")
//let myQuiz = Quiz(title: "Quiz 1", isPremium: nil)
let myQuiz: Quiz = Quiz(title: "Quiz 1", dateCreated: nil, isPremium: false)

print(myQuiz.title)


// ----------------------------------------------------


// "Immutable struct" = all "let" constants = NOT mutable = "cannot mutate it!"
struct UserModel {
    let name: String
    let isPremium: Bool
}

var user1: UserModel = UserModel(name: "Nick", isPremium: false)

// make the totally new object, but with the same user name and switch the isPremium
func markUserAsPremium() {
    print(user1)
    user1 = UserModel(name: user1.name, isPremium: true)
    print(user1)
}

//markUserAsPremium()

// ----------------------------------------------------

// "mutable struct"
// this version use the function that is not in the struct, but the below "UserModel4" put the function in the
struct
struct UserModel2 {
    let name: String
    var isPremium: Bool
}

var user2 = UserModel2(name: "Nick", isPremium: false)

func markUserAsPremium2() {
    print(user2)

    // "mutate" the struct
    user2.isPremium = true

    print(user2)
}
```

```
markUserAsPremium2()

// -----------------------------------------------------

// "immutable struct"
// And add the function in the struct
struct UserModel3 {
    let name: String
    let isPremium: Bool

    func markUserAsPremium(newValue: Bool) -> UserModel3 {
        UserModel3(name: name, isPremium: newValue)
    }
}

var user3: UserModel3 = UserModel3(name: "Nick", isPremium: false)
user3 = user3.markUserAsPremium(newValue: true)

// -----------------------------------------------------

// "mutable struct"

struct UserModel4 {
    let name: String
    private(set) var isPremium: Bool

    mutating func markUserAsPremium() {
        isPremium = true
    }

    mutating func updateIsPremium(newValue: Bool) {
        isPremium = newValue
    }
}

var user4 = UserModel4(name: "Nick", isPremium: false)
user4.markUserAsPremium()
user4.updateIsPremium(newValue: true)


// rather than using tuple, we can use struct
struct User5 {
    let name: String
    let isPremium: Bool
    let isNew: Bool
    //
    //
    //
    //
}
```

Enums.xcplaygroundpage/Contents.swift

```swift
import Foundation

// Enum is the same as Struct except we know all cases at runtime

struct CarModel {
    let brand: CardBrandOption
    let model: String
}

//struct CarBrand {
//    let title: String
//}


// Enums are stored in memory the same way as a Struct but we cannot mutate them
enum CardBrandOption {
    case ford, toyota, honda


    // u need the title for these brand
    var title: String {
        switch self {
        case .ford:
            return "Ford"
        case .toyota:
            return "Toyota"
        case .honda:
            return "Honda"
//        default:
//            return "Default"
        }
    }
}

/*The below function is created when there is no enum function
//var car1: CarModel = CarModel(brand: "Ford", model: "Fiesta")
//var car2: CarModel = CarModel(brand: "Ford", model: "Focus")
//var car3: CarModel = CarModel(brand: "Toyota", model: "Camry")
*/

/* The below function is also created before enum function
//var brand1 = CarBrand(title: "Ford")
//var brand2 = CarBrand(title: "Toyota")
//
//var car1 = CarModel(brand: brand1, model: "Fiesta")
//var car2 = CarModel(brand: brand1, model: "Focus")
//var car3 = CarModel(brand: brand2, model: "Camry")
 */

// the .variables are constrain to the case that clarify in the enum function
var car1 = CarModel(brand: .ford, model: "Fiesta")
var car2 = CarModel(brand: .ford, model: "Focus")
var car3 = CarModel(brand: .toyota, model: "Camry")

var car4 = CarModel(brand: .toyota, model: "Camry")

var fordBrand: CardBrandOption = .ford

print(fordBrand.title)
```

Classes.xcplaygroundpage/Contents.swift

```swift
import Foundation


// Classes are slow!
// Classes are stored in the Heap (memory)
// Objects in the Heap are Reference types
// Reference types point to an object in memory and update the object in memory


// All the data needed for some screen
class ScreenViewModel {
    let title: String
    private(set) var showButton: Bool

    // Same init as a Struct, except structs have implicit inits
    init(title: String, showButton: Bool) {
        self.title = title
        self.showButton = showButton
    }

    deinit {
        // runs as the object is being removed from memory
        // Structs do NOT have deinit!
    }

    func hideButton() {
        showButton = false
    }

    func updateShowButton(newValue: Bool) {
        showButton = newValue
    }
}


// Notice that we are using a "let", because:
// the object itself is not changing
// the data inside the object is changing
let viewModel: ScreenViewModel = ScreenViewModel(title: "Screen 1", showButton: true)
//viewModel.showButton = false
let value = viewModel.showButton

viewModel.hideButton()
viewModel.updateShowButton(newValue: false)
```

AccessControl.xcplaygroundpage/Contents.swift

```
import Foundation

// Rule of thumb:
// We want everything to be as private as possible!
// This makes your code easier to read/debug and is good coding practice!

struct MovieModel {
    let title: String
    let genre: MovieGenre
    private(set) var isFavorite: Bool

    func updateFavoriteStatus(newValue: Bool) -> MovieModel {
        MovieModel(title: title, genre: genre, isFavorite: newValue)
    }

    mutating func updateFavoriteStatus2(newValue: Bool) {
        isFavorite = newValue
    }
}

enum MovieGenre {
    case comedy, action, horror
}

class MovieManager {

    // public
    public var movie1 = MovieModel(title: "Avatar", genre: .action, isFavorite: false)

    // private
    private var movie2 = MovieModel(title: "Step Brothers", genre: .comedy, isFavorite: false)

    // read is public, set is private
    private(set) var movie3 = MovieModel(title: "Avenger", genre: .action, isFavorite: false)

    func updateMovie3(isFavorite: Bool) {
        movie3.updateFavoriteStatus2(newValue: isFavorite)
    }
}


let manager = MovieManager()

let someValue = manager.movie3

//manager.movie1 = manager.movie1.updateFavoriteStatus(newValue: true)
//manager.movie3.updateFavoriteStatus2(newValue: true)
manager.updateMovie3(isFavorite: true)
print(manager.movie3)



// Version 1
// We can GET and SET the value from outside the object.
// "too public"
let movie1 = manager.movie1
manager.movie1 = manager.movie1.updateFavoriteStatus(newValue: true)


// Version 2
// We can't GET or SET the value from outside the object.
// "cannot access"
//let movie2 = manager.movie2
//manager.movie2 = manager.movie2.updateFavoriteStatus(newValue: true)

// Version 3
// We can GET the value from outside the object, but we can't SET the value from outside the object.
// "best practice"
let movie3 = manager.movie3
//manager.movie3 = manager.movie3.updateFavoriteStatus(newValue: true)
manager.updateMovie3(isFavorite: true)



// Note: private & public are by far the most common but there are many others
//
// open
// public
// internal
// fileprivate
// private
```

```
//
// https://abhimuralidharan.medium.com/swift-3-0-1-access-control-9e71d641a56c
```

Arrays.xcplaygroundpage/Contents.swift

```
import Foundation


// Arrays, Sets


var myTitle: String = "Hello, world!"
var myTitle2: String = "Hello, world!!!"

// Tuple

func doSomething(value: (title1: String, title2: String)) {

}

doSomething(value: (myTitle, myTitle2))

// Custom data model
struct TitlesModel {
    let title1: String
    let title2: String
}

func doSomething(value: TitlesModel) {

}

doSomething(value: TitlesModel(title1: myTitle, title2: myTitle2))


// --------------------------


let apple: String = "Apple"
let orange: String = "Orange"

let fruits1: [String] = ["Apple", "Orange"]
let fruits2: [String] = [apple, orange]
let fruits3: Array<String> = [apple, orange]

let myBools: [Bool] = [true, false, true, true, true, false]

func doSomething(value: [String]) {

}


var fruitsArray: [String] = ["Apple", "Orange"]


let count = fruitsArray.count
let firstItem = fruitsArray.first
let lastItem = fruitsArray.last

if let firstItem = fruitsArray.first {
    // first item
}


//fruitsArray = fruitsArray + ["Banana", "Mango"]

//fruitsArray.append("Banana")
//fruitsArray.append("Mango")

fruitsArray.append(contentsOf: ["Banana", "Mango"])


// Count     = 1, 2, 3, 4
// Indexes   = 0, 1, 2, 3

fruitsArray[0]

if fruitsArray.indices.contains(4) {
    let item = fruitsArray[4]
}


let firstIndex = fruitsArray.indices.first
let lastIndex = fruitsArray.indices.last


//fruitsArray.append("Watermelon")
```

```swift
//fruitsArray.insert("Watermelon", at: 2)
fruitsArray.insert(contentsOf: ["Watermelon", "Tangerine"], at: 2)

if fruitsArray.indices.contains(1) {
    fruitsArray.remove(at: 1)
}

fruitsArray.removeAll()

print(fruitsArray)


struct ProductModel {
    let title: String
    let price: Int
}

var myProducts: [ProductModel] = [
    ProductModel(title: "Product 1", price: 50),
    ProductModel(title: "Product 2", price: 5),
    ProductModel(title: "Product 3", price: 1),
    ProductModel(title: "Product 4", price: 50),
    ProductModel(title: "Product 5", price: 4),
    ProductModel(title: "Product 6", price: 50),
    ProductModel(title: "Product 7", price: 2),
]


var finalFruits: [String] = ["Apple", "Orange", "Banana", "Apple"]

print(finalFruits)

var fruitsSet: Set<String> = ["Apple", "Orange", "Banana", "Apple"]

print(fruitsSet)
```

Dictionaries.xcplaygroundpage/Contents.swift

```
import Foundation

var finalFruits: [String] = ["Apple", "Orange", "Banana", "Apple"]

print(finalFruits)

let myFruit = finalFruits[1]

var fruitsSet: Set<String> = ["Apple", "Orange", "Banana", "Apple"]

print(fruitsSet)


var myFirstDictionary: [String : Bool] = [
    "Apple" : true,
    "Orange" : false
]


let item = myFirstDictionary["Banana"]



var anotherDictionary: [String : String] = [
    "abc" : "Apple",
    "def" : "Banana",
]

let item2 = anotherDictionary["abc"]

anotherDictionary["xyz"] = "Mango"

anotherDictionary.removeValue(forKey: "def")

print(anotherDictionary)


struct PostModel {
    let id: String
    let title: String
    let likeCount: Int
}

var postArray: [PostModel] = [
    PostModel(id: "abc123", title: "Post 1", likeCount: 5),
    PostModel(id: "def678", title: "Post 2", likeCount: 7),
    PostModel(id: "xyz987", title: "Post 3", likeCount: 217),
]

if postArray.indices.contains(1) {
    let item = postArray[1]
    print(item)
}

var postDict: [String:PostModel] = [
    "abc123" : PostModel(id: "abc123", title: "Post 1", likeCount: 5),
    "def678" : PostModel(id: "def678", title: "Post 2", likeCount: 7),
    "xyz987" : PostModel(id: "xyz987", title: "Post 3", likeCount: 217),
]

let myNewItem = postDict["def678"]
print(myNewItem)
```

ForLoops.xcplaygroundpage/Contents.swift

```
import Foundation

// Learn more:
// https://docs.swift.org/swift-book/documentation/the-swift-programming-language/controlflow/#For-In-Loops

//for x in 0..<50 {
//    print(x)
//}

let myArray = ["Alpha", "Beta", "Gamma", "Delta", "Epsilon"]


var secondArray: [String] = []

for item in myArray {
    print(item)

    if item == "Gamma" {
        secondArray.append(item)
    }
}

print(secondArray)

struct LessonModel {
    let title: String
    let isFavorite: Bool
}

let allLessons = [
    LessonModel(title: "Lesson 1", isFavorite: true),
    LessonModel(title: "Lesson 2", isFavorite: false),
    LessonModel(title: "Lesson 3", isFavorite: false),
    LessonModel(title: "Lesson 4", isFavorite: true),
]

var favoriteLessons: [LessonModel] = []

for lesson in allLessons {
    if lesson.isFavorite {
        favoriteLessons.append(lesson)
    }
}

print(favoriteLessons)


for (index, lesson) in allLessons.enumerated() {

    if index == 1 {
//        break
        continue
    }

    print("index: \(index) || lesson: \(lesson)")
}
```

FilterSortMap.xcplaygroundpage/Contents.swift

```
import Foundation


struct DatabaseUser {
    let name: String
    let isPremium: Bool
    let order: Int
}

var allUsers: [DatabaseUser] = [
    DatabaseUser(name: "Nick", isPremium: true, order: 4),
    DatabaseUser(name: "Emily", isPremium: false, order: 1),
    DatabaseUser(name: "Samantha", isPremium: true, order: 3),
    DatabaseUser(name: "Joe", isPremium: true, order: 10000),
    DatabaseUser(name: "Chris", isPremium: false, order: 2),
]


//var allPremiumUsers: [DatabaseUser] = allUsers.filter { user in
//    if user.isPremium {
//        return true
//    }
//
//    return false
//}

var allPremiumUsers: [DatabaseUser] = allUsers.filter { user in
    user.isPremium
}

var allPremiumUsers2: [DatabaseUser] = allUsers.filter({ $0.isPremium })


//for user in allUsers {
//    if user.isPremium {
//        allPremiumUsers.append(user)
//    }
//}
//print(allPremiumUsers)


var orderedUsers: [DatabaseUser] = allUsers.sorted { user1, user2 in
    return user1.order < user2.order
}

var orderedUsers2: [DatabaseUser] = allUsers.sorted(by: { $0.order < $1.order })

print(orderedUsers)


var userNames: [String] = allUsers.map { user in
    return user.name
}

var userNames2: [String] = allUsers.map({ $0.name })

print(userNames)
```

Protocols.xcplaygroundpage/Contents.swift

```
import Foundation

struct EmployeeModel: EmployeeHasAName {
    let title: String
    let name: String
}

protocol EmployeeHasAName {
    let name: String
}
```